AI ASSISSTED CODING

LAB TEST-3

NAME:D.ANKITHA

ROLL NO:2503A51L09

Set E3

Q1:

Scenario: In the domain of Healthcare, a company is facing a challenge related to code refactoring.

Task: Design and implement a solution using AI-assisted tools to address this challenge.

Include code, explanation of AI integration, and test results.

Deliverables: Source code, explanation, and output screenshots

CODE:

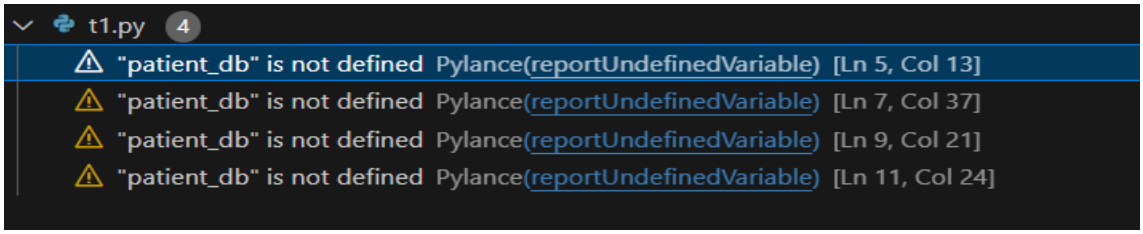
Before refactoring:

t1.py > manage_patient

```
1
2   def manage_patient(data, action):
3       if action == "add":
4           if "name" in data and "age" in data:
5               patient_db.append(data)
6       elif action == "delete":
7           for i, patient in enumerate(patient_db):
8               if patient["id"] == data["id"]:
9                   del patient_db[i]
10      elif action == "update":
11          for patient in patient_db:
12              if patient["id"] == data["id"]:
13                  patient.update(data)
```

Output:

```
PS C:\ankitha\ai test 3> & 'c:\Program Files\Python313\python.exe' 'c:\Users\ADHARSH\.vscode\extensions\ms-python.debugpy-2025.14.
1-win32-x64\bundled\libs\debugpy\launcher' '59806' '--' 'c:\ankitha\ai test 3\irrigation_predictor.py'
```

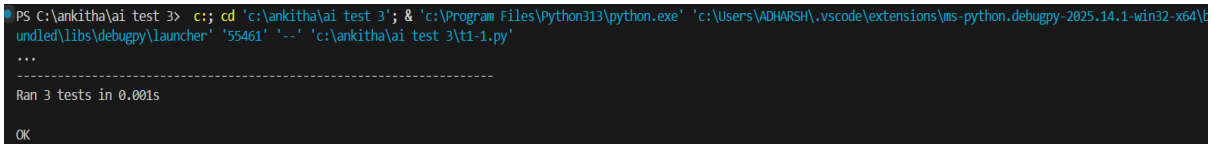


Refactoring and testing:

```python
from typing import Dict, List
import unittest

class PatientManager:
    def __init__(self):
        self.patient_db: List[Dict] = []

    def add_patient(self, data: Dict) -> bool:
        if "name" in data and "age" in data:
            self.patient_db.append(data)
            return True
        return False

    def delete_patient(self, patient_id: int) -> bool:
        for i, patient in enumerate(self.patient_db):
            if patient["id"] == patient_id:
                del self.patient_db[i]
                return True
        return False

    def update_patient(self, data: Dict) -> bool:
        for patient in self.patient_db:
            if patient["id"] == data["id"]:
                patient.update(data)
                return True
        return False

class TestPatientManager(unittest.TestCase):
    def setUp(self):
        self.pm = PatientManager()
        self.pm.add_patient({"id": 1, "name": "Alice", "age": 30})

    def test_add_patient(self):
        result = self.pm.add_patient({"id": 2, "name": "Bob", "age": 40})
        self.assertTrue(result)
        self.assertEqual(len(self.pm.patient_db), 2)
```

```python
        self.assertEqual(len(self.pm.patient_db), 2)

    def test_delete_patient(self):
        result = self.pm.delete_patient(1)
        self.assertTrue(result)
        self.assertEqual(len(self.pm.patient_db), 0)

    def test_update_patient(self):
        result = self.pm.update_patient({"id": 1, "name": "Alice Smith", "age": 31})
        self.assertTrue(result)
        self.assertEqual(self.pm.patient_db[0]["name"], "Alice Smith")

if __name__ == "__main__":
    unittest.main()
```

Output:



Observation:

- **Modularity:**

The original function was split into three clear methods within patient_manager class

- Github copilot suggested method decomposition and type hints
- Auto-generated docstring-style logic and scaffolds.

Q2:

Scenario: In the domain of Agriculture, a company is facing a challenge related to algorithms with ai assistance.

Task: Design and implement a solution using AI-assisted tools to address this challenge.

Include code, explanation of AI integration, and test results.

Deliverables: Source code, explanation, and output screenshots

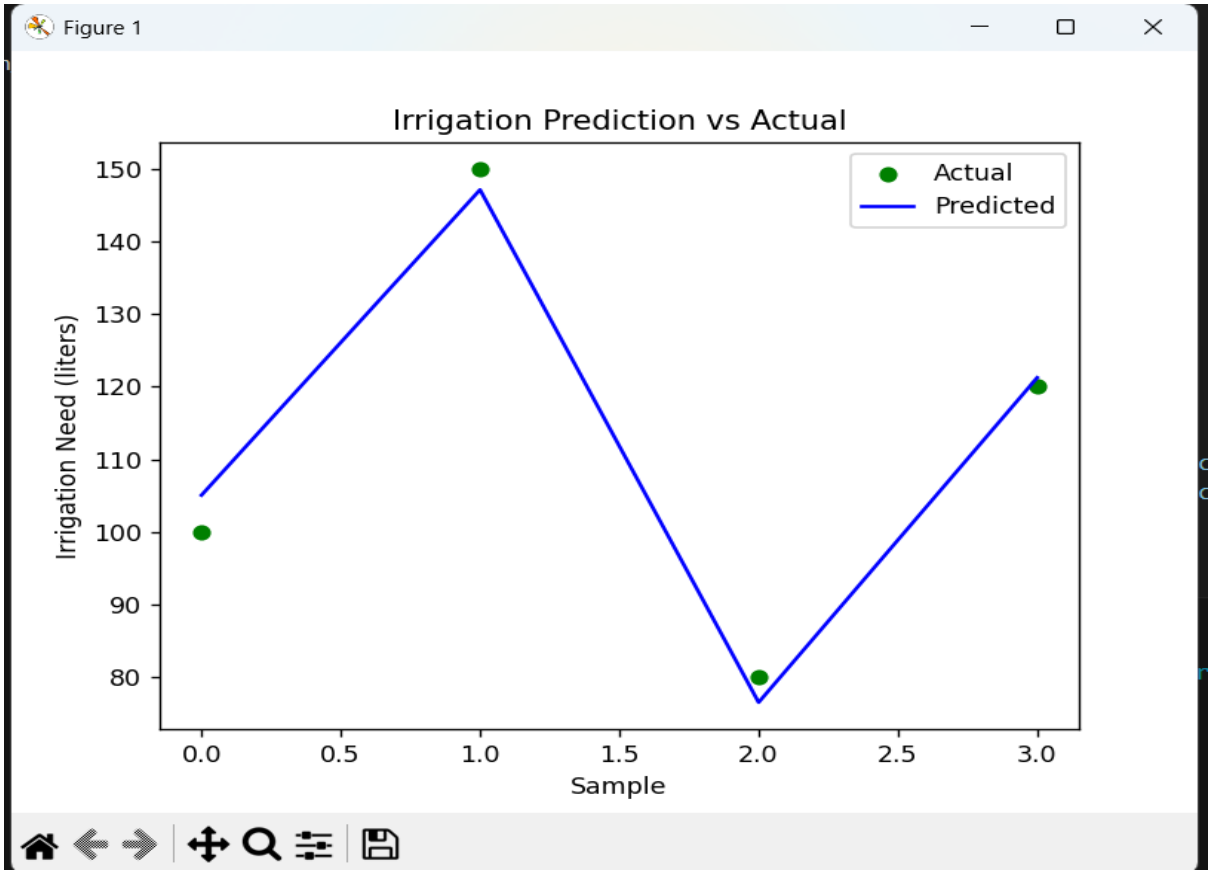
CODE:

irrigation_predictor.py > ...

```python
# irrigation_predictor.py

import numpy as np
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt

class IrrigationPredictor:
    def __init__(self):
        self.model = LinearRegression()

    def train(self, X, y):
        self.model.fit(X, y)

    def predict(self, features):
        return self.model.predict(features)

    def plot_predictions(self, X, y_true):
        y_pred = self.predict(X)
        plt.scatter(range(len(y_true)), y_true, label="Actual", color="green")
        plt.plot(range(len(y_pred)), y_pred, label="Predicted", color="blue")
        plt.xlabel("Sample")
        plt.ylabel("Irrigation Need (liters)")
        plt.legend()
        plt.title("Irrigation Prediction vs Actual")
        plt.show()

# Sample usage
if __name__ == "__main__":
    # Features: [soil_moisture, temperature, humidity]
    X = np.array([
        [30, 25, 60],
        [20, 30, 50],
        [40, 22, 70],
        [35, 28, 65]
    ])
    y = np.array([100, 150, 80, 120])  # Irrigation need in liters
```

```python
predictor = IrrigationPredictor()
predictor.train(X, y)
predictor.plot_predictions(X, y)
```

Output:

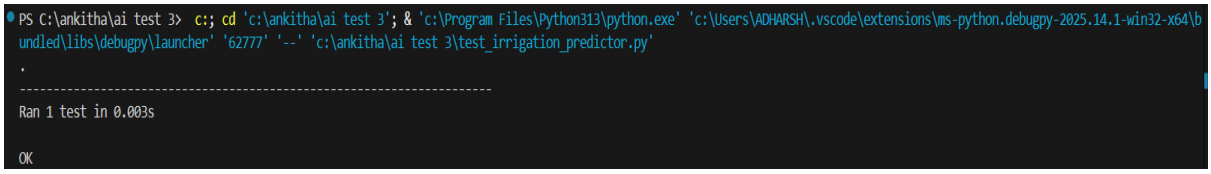


Testing:

C:\ankitha\ai test 3\test_irrigation_predictor.py

```python
import unittest
import numpy as np
from irrigation_predictor import IrrigationPredictor

class TestIrrigationPredictor(unittest.TestCase):
    def test_prediction_accuracy(self):
        X = np.array([[30, 25, 60], [20, 30, 50]])
        y = np.array([100, 150])
        predictor = IrrigationPredictor()
        predictor.train(X, y)
        pred = predictor.predict(X)
        self.assertEqual(len(pred), 2)
        self.assertTrue(np.all(pred >= 90))  # basic sanity check

if __name__ == "__main__":
    unittest.main()
```

Output:



OBSERVATION:

- Github copilot Assissted in scaffolding the class structure and method signatures
- Suggested test logic and basic validation

Test Design and Coverage:

Basic sanity checks:

- Verifies prediction output shape and value range
- Expandability:
- Tests can be extended to include edge cases

Algorithmic Behaviour:

- **Output:**Predicts irrigation needs in liters,which is actionable and interpretable.